(Optional) PDO Troubleshooting and MySQLi

Start Here If Something Fails

Troubleshooting is the first part of this document.

Check these in order:

1. PDO MySQL driver
2. PHP and MySQL server status
3. Connection information: database, username, and password

Check the PDO MySQL Driver

If `new PDO("mysql:...")` reports **could not find driver**:

```
php -m | grep -i pdo
php -i | grep "PDO drivers"
```

On WSL2/Ubuntu, install the PHP MySQL package if it is missing:

```
sudo apt install php-mysql
```

Restart the PHP server after installing it.

Check the Two Servers

If PHP cannot connect to MySQL, check each server separately.

1. Is the PHP development server running?
2. Is the MySQL service running?
3. Does the database name match `studentdb`?
4. Do the username and password match your MySQL account?

WSL2/Ubuntu:

```
sudo service mysql status
```

macOS:

```
brew services list
```

Optional PDO Reference

After the connection works, these topics are optional. They are useful but not required for the core CRUD pattern:

- Prepared statements
- `ATTR_EMULATE_PREPARES`
- Safe HTML output
- MySQLi

Keep using the core pattern after troubleshooting:

```
SQL → prepare → execute → fetch when reading
```

Why Use Prepared Statements?

A query has two different parts: the SQL structure and the changing value.

- The value may come from a user, such as a student ID entered in a form.

```
SQL structure:  SELECT * FROM students WHERE id = :id
Value:          :id receives 7 (for example)
```

`:id` is a named empty place in the SQL structure. PDO fills it with the value only when `execute()` runs.

Prepared statements keep these parts separate.

- This prevents a value from being treated as SQL code and lets PDO safely handle values such as names containing quotes.

For example, if a form provides the ID `7`:

```
$stmt = $pdo->prepare(
    "SELECT * FROM students WHERE id = :id"
);
$stmt->execute([':id' => 7]);
```

`prepare()` creates a prepared statement object.

- With native prepares, PDO sends the SQL structure to MySQL for preparation; with emulated prepares, PDO manages the preparation behavior itself.
- `execute()` supplies `7` as data for that placeholder.

`ATTR_EMULATE_PREPARES => false`

This is one optional setting added when PHP creates the `$pdo` connection:

```
$pdo = new PDO($dsn, $username, $password, [
    PDO::ATTR_EMULATE_PREPARES => false,
]);
```

Both choices (true/false) run the same PHP code, but they send the query to MySQL differently:

Setting	What happens	Advantage	Limitation
`true`	PDO fills `:id` with `7`, then sends complete SQL.	Default; simple for the first CRUD examples.	MySQL does not prepare the SQL itself.
`false`	PDO sends SQL structure and `7` separately.	MySQL parses and prepares the SQL.	Can behave differently for some advanced queries.

When `false` Is Useful: Import Many Rows

For example, an application imports 10,000 students using the same `INSERT` command:

```
$stmt = $pdo->prepare(
    "INSERT INTO students (name) VALUES (:name)"
);
$stmt->execute([':name' => 'Alice']);
$stmt->execute([':name' => 'Bob']);
// Repeat for the remaining students.
```

- With `false`, MySQL prepares the `INSERT` structure **once** and receives a new name for each `execute()`.
- This can be useful for large repeated operations, but it is not needed for our small CRUD example.

Why the Core Examples Use `true`

The core code does not set this option, so `pdo_mysql` uses its default, `true`.

- With `true`, PDO fills each placeholder with the value from `execute()` before it sends the **complete SQL** to MySQL.
- PDO still treats that value as data, so it cannot change the SQL structure.
- We use this default to focus first on the `prepare()` and `execute()` pattern, not connection configuration.

Safe HTML Output

Database safety and browser output safety are different concerns.

- Prepared statements protect the SQL query when PHP saves or reads a value.
- They do not control how the browser displays that value.

For example, a student name stored in MySQL might contain `<b>Alice</b>`; Without escaping, the browser treats it as HTML instead of showing the characters as a name.

```
echo htmlspecialchars($student['name'], ENT_QUOTES, 'UTF-8');
```

- `htmlspecialchars()` converts HTML-special characters to safe text before output.
- This is important for web applications, but it is separate from learning the PDO CRUD pattern.

MySQLi Reference

MySQLi is another PHP interface for MySQL.

- It only supports MySQL/MariaDB.
- It uses `?` placeholders and a separate `bind_param()` step.
- You may encounter it in existing PHP code.
- This course teaches and uses PDO instead.

See `insert_mysqli_optional.php` (first INSERT) and `crud_mysqli_optional.php` (full CRUD) in `module1/code/2_Connect_PHP_with_MySQL/` for working examples.

```
Required: PDO
Reference only: MySQLi
```

PDO and MySQLi Syntax

PDO:

```php
$stmt = $pdo->prepare(
    "SELECT * FROM students WHERE id = :id"
);
$stmt->execute([':id' => $aliceId]);
$student = $stmt->fetch();
```

MySQLi reference:

```php
$stmt = $conn->prepare(
    "SELECT * FROM students WHERE id = ?"
);
$stmt->bind_param("i", $aliceId);
$stmt->execute();
```